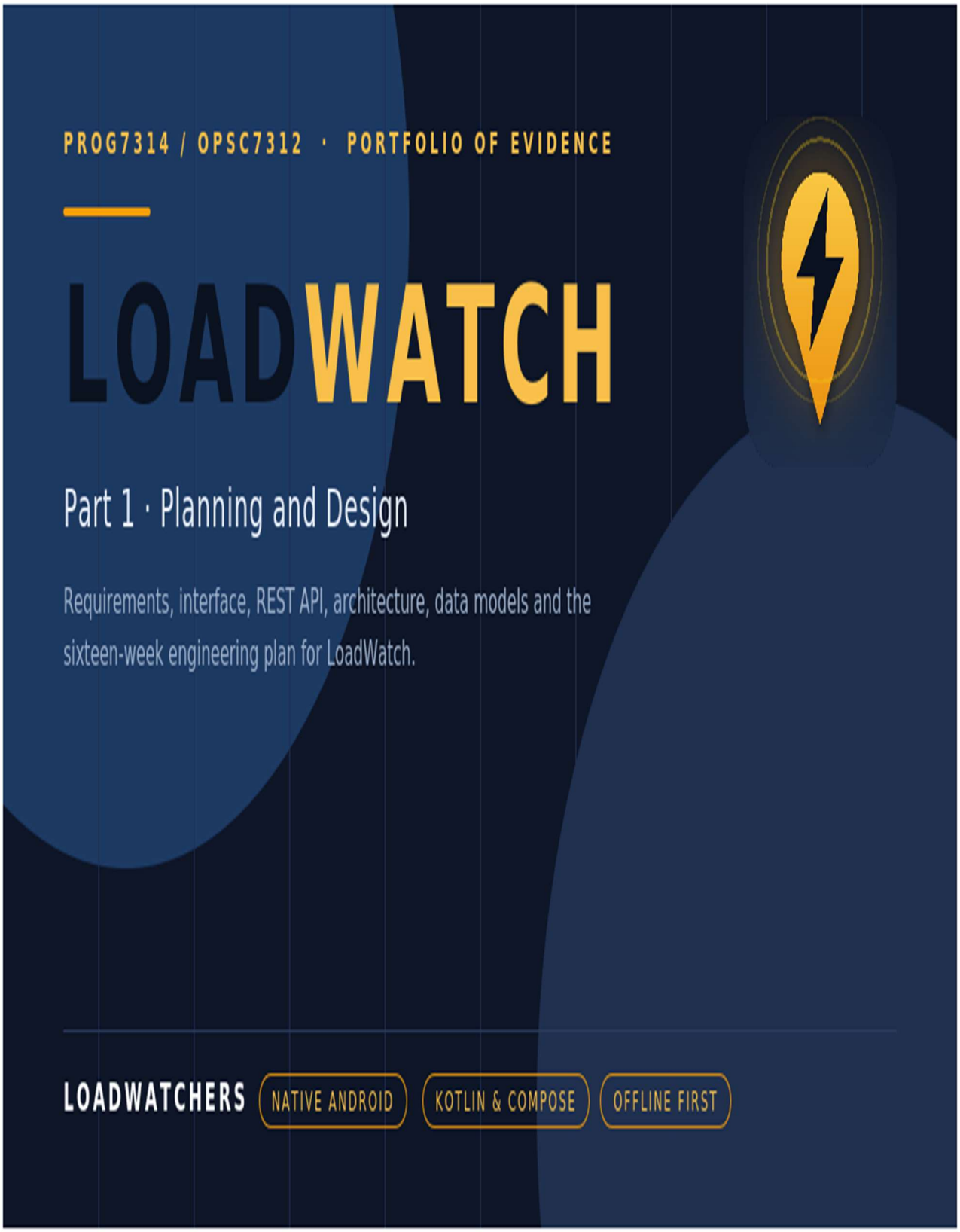


Group	Loadwatchers - 12	**Application**	LoadWatch
Members	Muaaz (Group Leader), Monwabisi, Jadey, Rahil		
Modules	Programming 3D (PROG7314) / Open-Source Coding Intermediate (OPSC7312)		
Date	24 August 2026	**Word Count**	2497 words excluding diagrams, headings and tables

1. Introduction

This document converts the findings of the LoadWatch Research Report into an implementable design. That report established that the three comparators publish timetables competently, but none reconciles the published schedule with what users experience and none converts a timestamp into remaining time. LoadWatch is therefore designed as a narrow, offline-capable Android application in which the schedule, a live countdown and structured community confirmation sit on a single screen.

Sections 2 and 3 fix what the product is and what it must do. Section 4 specifies every screen and the navigation between them. Section 5 defines the REST API, the development stack and synchronisation, Section 6 gives the UML architecture views, and Section 7 defines the data that must be persisted. Section 8 schedules sixteen weeks of engineering work across the four members of Loadwatchers, and Section 9 closes by confirming that nothing in the Final POE list requires Part 2 code to be rewritten.

2. Application Overview

2.1 Name, purpose and users

The application is named **LoadWatch**. The name states the product in two words: it watches load, both scheduled load shedding and unplanned load reduction, for the areas a user cares about. It is a native Android application written in Kotlin with Jetpack Compose, aimed at South African households, small businesses and students who need to know whether power is on.

Three user types are recognised. A casual user reads one area and leaves. A committed user saves several areas, sets a notification lead time and receives alerts. A contributing user confirms what is actually happening in their area, which feeds the per-area accuracy score. The design does not force the later roles onto the first: signing in is required only for contribution, favourites and settings synchronisation, so a public schedule is readable immediately after install.

2.2 Application icon concept

The icon is a location pin whose interior is a lightning bolt, drawn on a deep slate ground with an amber bolt and a thin amber ring. The pin carries the meaning "this place", the bolt carries "electricity" and the ring carries "watching". Amber was chosen over red because the product reports a state rather than an emergency, and over green because green implies supply is on, which is only sometimes true. The mark reads at 48 dp on a launcher and survives monochrome theming on Android 13 and above.

2.3 Innovative features

Five features define the product. Each is innovative in the specific sense the rubric requires: it is either absent from all three comparators or restructured so that it produces data none of them can produce. The live countdown converts a timestamp into remaining time. Structured confirmation replaces free-form chat with a two-state event the backend can aggregate. The per-area accuracy score turns those events into a public trust signal. Side-by-side comparison answers the multi-location question on one screen. Offline-first operation keeps the product usable during the very event it describes. Table 1 states each feature, the gap it closes and the evidence behind it.

Feature	Phase	Why it is innovative in this category
Live countdown	Part 2	No comparator converts a schedule timestamp into remaining time. The countdown is computed on-device from ISO-8601 boundaries supplied by the API, so it adds no network cost yet turns passive information into an action cue.
Community outage confirmation	Part 2	ESP proves demand for community signal but collects it as free-form chat, which cannot be aggregated. LoadWatch collects a two-state enumerated event, so the backend can compute a per-area accuracy score. It is the first structured measurement of schedule truth in this set.
Favourites and compare	Part 2	Competitors let a user follow several areas but never place them side by side with their next event and countdown. Comparison is where the multi-area feature actually pays off.
Stage-change alert	Final POE	The backend recalculates every affected schedule when a stage changes, then pushes a deep link into the recalculated screen. Recalculation stays authoritative on the server; the client stays thin.
Historical outage calendar	Final POE	Overlaying past scheduled events with past community confirmations produces a visible record of how often the published schedule matched reality, which is the long-run evidence behind the accuracy score.

Table 1. The five user-defined features and their innovation justification.

3. Detailed Requirements Specification

3.1 Functional requirements

Forty functional requirements are grouped into ten capabilities. Each carries an identifier, a testable acceptance criterion and a phase, so Part 2 and Final POE scope are separated before any code is written. Table 2 gives the requirements themselves.

Authentication (FR-AUTH, five requirements): The user registers and signs in through Google single sign-on, the session survives relaunch, unauthenticated access to user-scoped data is refused by both client and API, sign-out clears tokens and user cache without clearing the public area catalogue, and biometric re-entry protects a signed-in session in the Final POE.

Area management (FR-AREA, four requirements): The user searches the area catalogue by suburb or municipality, selects an area during onboarding, changes the active area at any time, and sees the supplier and municipality alongside the area name so that two similarly named suburbs cannot be confused.

Schedule retrieval (FR-SCHED, four requirements): The client requests the selected area's schedule window from the API, every event shows stage, date, start and end in device local time, Home derives whether supply is expected on or off, and data older than a set threshold is flagged as stale.

Countdown (FR-COUNT, three requirements): The countdown ticks locally from server-supplied boundaries without repeated API calls, it distinguishes time until power off from time until power returns, and on expiry it rolls forward to the next event without user action.

Community confirmation (FR-REP, five requirements): A signed-in user submits a structured POWER_OFF or POWER_BACK_ON event carrying area, user, type, timestamp, a client-generated identifier and sync status. Repeat submissions inside a configurable window are de-duplicated, the API aggregates reports into a per-area accuracy score, and a report made without connectivity queues locally as pending.

Favourites and comparison (FR-FAV, three requirements): The user saves several areas, removes them, and compares next-event times across saved areas on one screen rather than switching context repeatedly.

Settings (FR-SET, five requirements): The user sets notification lead time, the event types that raise alerts, interface language and theme, and these preferences persist locally and synchronise to the account.

Notifications (FR-NOTIF, four requirements): Push messages are delivered through Firebase Cloud Messaging, the device registers a token per user, stage changes trigger a server-side recalculation before any send, and tapping a notification opens the relevant area rather than the launcher screen.

Offline operation (FR-OFF, four requirements): Cached schedules render without connectivity, the interface states plainly that data is cached, queued reports are retried with backoff when connectivity returns, and a conflict between a queued report and server state resolves in favour of the server with the outcome shown to the user.

History and storage (FR-HIST, FR-MEDIA, FR-NOSQL): The Final POE adds a past-month outage calendar, blob-served image assets and NoSQL historical documents, all reached through the API.

ID	Requirement	Acceptance criterion	Phase
FR-AUTH-01	The user shall register and sign in through single sign-on using Firebase Authentication with Google [13].	A first-time Google account reaches Home without a password form; a UserProfile row exists.	Part 2
FR-AUTH-02	The application shall persist authentication state and restore the session on relaunch.	After force-stop and relaunch, the user is not prompted to sign in again.	Part 2
FR-AUTH-03	Unauthenticated access to user-scoped data shall be refused by the client and the API.	A request without a valid ID token returns HTTP 401.	Part 2
FR-AUTH-04	The user shall be able to sign out, clearing tokens and user-scoped cache but not the public area catalogue.	After sign out the app returns to Screen 02 and no user data is readable.	Part 2
FR-AUTH-05	The application shall re-authorise sensitive access using AndroidX BiometricPrompt with device-credential fallback [15].	With biometrics enrolled and enabled, resuming after timeout shows the system prompt; declining returns to a locked state.	Final POE
FR-AREA-01	The user shall search the area catalogue by suburb name with results returned progressively as text is entered.	Typing three characters returns matching areas within two seconds on a 3G connection.	Part 2
FR-AREA-02	Each area result shall display its municipality and supply context to disambiguate similarly named suburbs.	Two areas sharing a name are distinguishable without opening either.	Part 2
FR-AREA-03	The selected default area shall persist locally and remotely for the signed-in user.	Selecting an area on one device makes it default after sign-in on another.	Part 2
FR-AREA-04	An unknown or unresolvable area shall produce a recoverable error state offering retry or reselection.	No crash, no blank screen; a labelled action is always present.	Part 2
FR-SCHED-01	The client shall request the selected area's schedule window from the API.	GET /api/schedules returns events for the requested range.	Part 2
FR-SCHED-02	Each schedule event shall display stage, date, start time and end time in device local time.	Displayed times match the API instant after time-zone conversion.	Part 2
FR-SCHED-03	Home shall derive whether supply is currently expected on or off and identify the next relevant event.	Within an event window the state reads OFF; outside it reads ON with the next start shown.	Part 2
FR-SCHED-04	The application shall display when schedule data was last refreshed and flag data older than six hours as stale.	A stale badge appears once the threshold passes.	Part 2
FR-COUNT-01	The countdown shall tick locally from server-supplied boundaries without repeated API calls.	With the network disabled the countdown continues to run correctly.	Part 2
FR-COUNT-02	The countdown shall distinguish time until power off from time until power returns.	The label and accent colour change at the event boundary.	Part 2
FR-COUNT-03	On expiry the countdown shall roll forward to the next event without user action.	At boundary crossing the target advances within one tick.	Part 2

FR-REP-01	A signed-in user shall submit a structured confirmation of POWER_OFF or POWER_BACK_ON for an area.	Both actions create an OutageReport with the correct enumerated eventType.	Part 2
FR-REP-02	A report shall carry areaId, userId, eventType, reportedAt, clientCreatedAt and syncStatus.	All fields are non-null on persistence.	Part 2
FR-REP-03	Repeat submissions by the same user for the same area and event within a configurable window shall be de-duplicated.	A second identical submission inside the window returns HTTP 409 and does not double-count.	Part 2
FR-REP-04	The API shall aggregate reports into a per-area accuracy score with a sample size.	GET /api/areas/{id}/accuracy returns a score between 0 and 1 with sampleSize.	Part 2
FR-REP-05	A report created without connectivity shall enter a local queue and display a pending state to the user.	In aeroplane mode the report is accepted by the UI and shown as queued.	Final POE
FR-FAV-01	The user shall save multiple areas as favourites with an optional label such as Home or Work.	Saved areas survive relaunch and reappear after reinstall and sign-in.	Part 2
FR-FAV-02	The compare screen shall show each favourite with its next event and countdown on one screen.	Three favourites render without scrolling on a 6-inch device.	Part 2
FR-FAV-03	Removing a favourite shall not delete the underlying public area or its schedule.	The area remains searchable after removal.	Part 2
FR-SET-01	The settings screen shall allow the user to change the default area.	The change is reflected on Home without restart.	Part 2
FR-SET-02	The user shall set a notification lead time between 5 and 120 minutes.	Values outside the range are rejected with an inline message.	Part 2
FR-SET-03	The user shall enable or disable notifications independently of lead time.	Disabling suppresses delivery while retaining the stored lead time.	Part 2
FR-SET-04	The user shall select an application language.	The selection persists and applies on next composition.	Part 2
FR-SET-05	All settings input shall be validated before persistence, with confirmation or an inline error.	No invalid value reaches the API; no crash on malformed input.	Part 2
FR-NOTIF-01	The application shall receive push notifications through Firebase Cloud Messaging [14].	A test message from the API reaches a physical device.	Final POE
FR-NOTIF-02	The API shall schedule a pre-outage alert using each user's stored lead time.	A user with a 30-minute lead time is notified 30 minutes before the event start.	Final POE
FR-NOTIF-03	A backend stage change shall recalculate affected schedules and publish to eligible users only.	Users of unaffected areas receive nothing.	Final POE
FR-NOTIF-04	Tapping a notification shall deep-link to the relevant area or schedule screen.	The correct screen opens with the correct areaId from a cold start.	Final POE
FR-OFF-01	The latest valid schedule for saved areas shall remain readable from Room when the network is	In aeroplane mode Home renders cached data with a stale indicator.	Final POE

	unavailable [12].		
FR-OFF-02	Queued writes shall be stored with an explicit synchronisation state.	Each queued item shows PENDING, SYNCED, RETRYING or REJECTED.	Final POE
FR-OFF-03	A connectivity observer shall trigger synchronisation on reconnection.	Restoring the network drains the queue without user action.	Final POE
FR-OFF-04	Transient failures shall retry with exponential backoff; permanent validation failures shall be marked rejected and surfaced.	User data is never silently discarded.	Final POE
FR-I18N-01	The interface shall be available in English, isiZulu and Afrikaans using Android string resources.	values-zu and values-af contain no untranslated keys.	Final POE
FR-HIST-01	The history screen shall present the previous month of scheduled events and community confirmations by area.	Days with a mismatch between scheduled and observed state are visually distinct.	Final POE
FR-MEDIA-01	Production assets and any user-supplied images shall be served from cloud blob storage rather than bundled.	An asset URL resolves from storage, not from the APK.	Final POE
FR-NOSQL-01	High-volume append-only records shall be written to and read from a NoSQL store.	A written document is retrievable through the API within one second.	Final POE

Table 2. Functional requirements. 40 identifiers across 10 capability groups, phased across Part 2 and the Final POE.

3.2 Non-functional requirements

Ten non-functional requirements constrain how the application behaves rather than what it does, and each carries a target that can be measured during the testing gates in Section 8. They cover modularity, reliability, performance, offline capability, security, accessibility, localisation, maintainability, observability and release readiness. Modularity is the load-bearing one: because no feature package imports another, four members can build simultaneously and the Final POE features attach without disturbing Part 2 code.

ID	Quality	Requirement and measurable target
NFR-01	Modularity	Each feature owns its package; no feature package imports another feature package. Verified by dependency review at each pull request.
NFR-02	Reliability	Invalid input and failed network calls produce recoverable UI states. Target: zero unhandled exceptions across the demonstration script.
NFR-03	Performance	Cached content renders without waiting for the network; cold start to first meaningful paint under 2 seconds on a mid-range device.
NFR-04	Security	User-scoped endpoints require a verified identity token. No API keys or secrets in source control; configuration is injected at build or runtime.
NFR-05	Testability	Repositories and business rules are unit-testable without Android instrumentation. Target: unit tests for every repository and the countdown calculator, run by CI on every push.
NFR-06	Usability	The current state, next event and countdown are visible on Home without scrolling or navigation.
NFR-07	Accessibility	Minimum 4.5:1 text contrast, content descriptions on all actionable elements, and no status conveyed by colour alone.
NFR-	Observability	Android Log statements mark lifecycle, authentication, API, Room and

08		synchronisation transitions, as required for Part 2 [1].
NFR-09	Data economy	Schedule refresh transfers only the requested window. Adopted from the low data-usage posture identified in the Research Report.
NFR-10	Localisation readiness	No user-facing string is hard-coded in Kotlin or Compose source, from Part 2 onward, so that FR-I18N-01 is additive.

Table 3. Non-functional requirements with measurable targets.

3.3 Traceability matrix

Table 4 traces every capability required by the brief through to the requirement identifiers that satisfy it, the screens that expose it, the endpoints that serve it, the member who owns it and the submission it belongs to. It exists so that no mandated capability can be lost between documents and so that ownership is unambiguous before the first sprint begins.

Mandated capability	Requirements	Screens	Endpoints	Owner	Marked in
SSO registration and login	FR-AUTH-01 to 04	S01, S02	Firebase Auth, GET /users/{id}/settings	Jadey	Part 2
Settings screen	FR-SET-01 to 05	S09, S05	GET and PUT /users/{id}/settings	Jadey	Part 2
Own REST API plus database	FR-AREA-01 to 04, FR-SCHED-01 to 04	S03, S05, S06	/areas, /schedules, /stages/current	Muaaz	Part 2
User-defined feature 1	FR-COUNT-01 to 03	S04	GET /schedules/next	Monwabisi	Part 2
User-defined feature 2	FR-REP-01 to 04	S07	POST /outage-reports, GET /areas/{id}/accuracy	Jadey	Part 2
User-defined feature 3	FR-FAV-01 to 03	S08	/users/{id}/favourites	Monwabisi	Part 2
Biometric authentication	FR-AUTH-05	S10	local, no endpoint	Rahil	Final POE
Offline mode with sync	FR-OFF-01 to 04, FR-REP-05	S13	POST /sync/outage-reports	Jadey	Final POE
Real-time notifications	FR-NOTIF-01 to 04	S11	POST /notifications/registrations	Rahil	Final POE
Multi-language support	FR-I18N-01, NFR-10	all	none	Rahil	Final POE
User-defined feature 4	FR-NOTIF-03	S11	internal stage-change event	Rahil	Final POE
User-defined feature 5	FR-HIST-01	S12	GET /areas/{id}/history	Rahil	Final POE
Blob storage and NoSQL	FR-MEDIA-01, FR-NOSQL-01	S12, assets	storage-backed asset and history reads	Muaaz	Final POE

Table 4. Traceability matrix: capability to requirement to screen to endpoint to owner to submission.

4. User Interface and Navigation Design

4.1 Visual system

The interface uses Material 3 with a dark-first palette: a slate surface, amber for state and action, green for supply on and red for supply off. Type is a single sans-serif family at four sizes, and the countdown

is the largest element on Home because it carries the most information. Colour is never the only carrier of meaning; every state is also labelled in words, which satisfies the accessibility requirement and keeps the product usable in bright outdoor light.

4.2 Screen catalogue and functional descriptions

Fourteen screens are specified and all fourteen are drawn in Figure 2. Screens 01 to 09 are Part 2 scope; screens 10 to 14 are Final POE surfaces designed now so that the navigation graph does not change later.

S01 Splash resolves session and cached settings and routes without showing the user a decision. **S02 SSO Login** offers Google sign-in and explains why an account is needed. **S03 Area Onboarding** is a search-first list that resolves a suburb to an area with its municipality and supplier visible, so the dominant onboarding error is designed out.

S04 Home Dashboard is the hub. It carries the area name, the current supply state, the live countdown, the next event and an accuracy badge for the area, with entry points to schedule, confirmation and favourites. Everything a casual user needs is on this one screen. **S05 Area Selector** switches the active area; **S06 Schedule** lists upcoming events grouped by day with stage badges, the lesson taken from Load Shedding Notifier.

S07 Community Confirmation is a two-button screen: power is off, or power is back. It shows what was submitted, whether it is queued or accepted, and how it moved the area's accuracy score, which is the visible lifecycle that ESP lacks. **S08 Favourites and Compare** lists saved areas with their next event times side by side. **S09 Settings** holds lead time, alert types, language and theme.

The Final POE screens are **S10 Biometric Unlock**, **S11 Stage-Change Detail**, **S12 History Calendar**, **S13 Offline and Sync**, showing the queue and its retry state, and **S14 Error and Empty States**, which is specified as a screen rather than left to chance because reliability is a measured requirement. Table 5 gives each screen's purpose, its interactive elements and the requirements it satisfies.

Two decisions run through the catalogue. First, every screen states its data freshness: if a schedule was read from cache the screen says so, because a user acting on stale times is worse served than a user told to refresh. Second, no screen depends on a network round trip to render; each opens from Room and updates when the network answers, which is why the interface behaves identically during an outage and outside one.

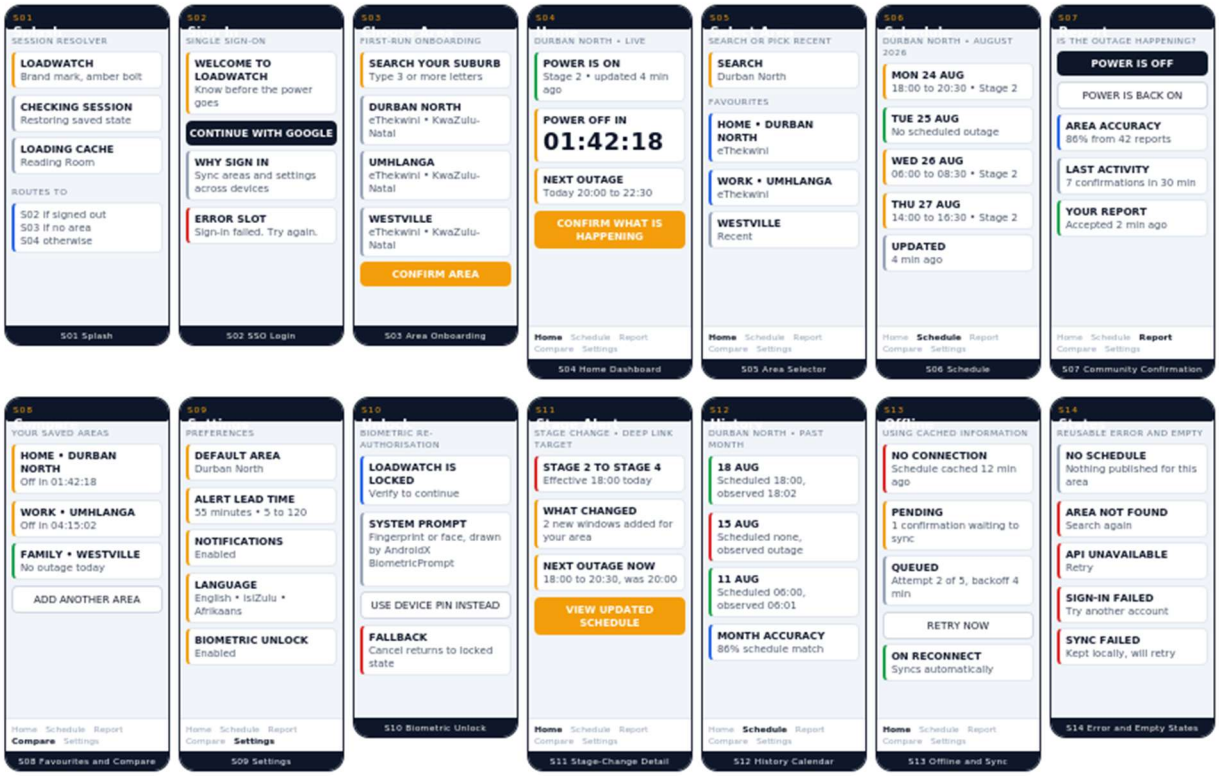


Figure 2. Mockups for all fourteen LoadWatch screens. Screens 10 to 14 are Final POE surfaces designed now.

ID	Screen	Purpose	Interactive elements	Requirements
S01	Splash	Resolve session and cached settings, then route without showing a decision to the user.	None; automatic routing only.	FR-AUTH-02
S02	SSO Login	Register or sign in with a Google account and explain why an account is needed.	Google sign-in button, privacy link, inline error slot.	FR-AUTH-01, 03
S03	Area Onboarding	Establish the default area on first run using search with disambiguating context.	Search field, result list, confirm button.	FR-AREA-01 to 04
S04	Home Dashboard	Answer the primary question: supply state, countdown, next event, one-tap report.	Countdown, report button, bottom navigation, pull to refresh.	FR-SCHED-03, FR-COUNT-01 to 03
S05	Area Selector	Switch the viewed area or add a favourite from search, recents or favourites.	Search field, favourite chips, result rows, add and remove.	FR-AREA-01, FR-FAV-01, 03
S06	Schedule	Show the upcoming window as a dated timeline with stage and time blocks.	Day rows, range selector, stale-data badge, refresh.	FR-SCHED-01, 02, 04
S07	Community Confirmation	Capture a structured observation and show its effect on area accuracy.	Two primary actions, accuracy card, submission status card.	FR-REP-01 to 05
S08	Favourites and Compare	Compare next event and countdown across saved areas on one screen.	Area cards, label edit, remove, add area.	FR-FAV-01 to 03

S09	Settings	Own the application's behaviour: area, lead time, notifications, language, biometrics.	Area picker, lead-time stepper, toggles, language list.	FR-SET-01 to 05
S10	Biometric Unlock	Re-authorise a returning session without repeating full sign-in.	System biometric prompt, device-credential fallback, cancel.	FR-AUTH-05
S11	Stage-Change Detail	Explain what changed after a stage transition and route to the updated schedule.	Change summary, view-schedule action; deep-link target.	FR-NOTIF-03, 04
S12	History Calendar	Show the past month with scheduled against observed events and the resulting accuracy.	Day rows, area switch, month accuracy summary.	FR-HIST-01, FR-NOSQL-01
S13	Offline and Sync	Make the offline state and the queue visible instead of failing silently.	Retry action, per-item queue state, backoff detail.	FR-OFF-01 to 04
S14	Error and Empty States	Provide one reusable component set so every failure has a labelled recovery action.	Retry, change area, sign in again, dismiss.	FR-AREA-04, FR-SET-05, NFR-02

Table 5. Functional description of every screen with its interactive elements and the requirements it satisfies.

4.3 Navigation wireflow

Navigation is a single graph with Home as the hub, reached through a splash and login wrapper and left only by explicit action. No destination is more than two taps from Home, and the Final POE screens attach to existing nodes rather than introducing a second hierarchy, which is what allows those features to be added without redrawing the graph.

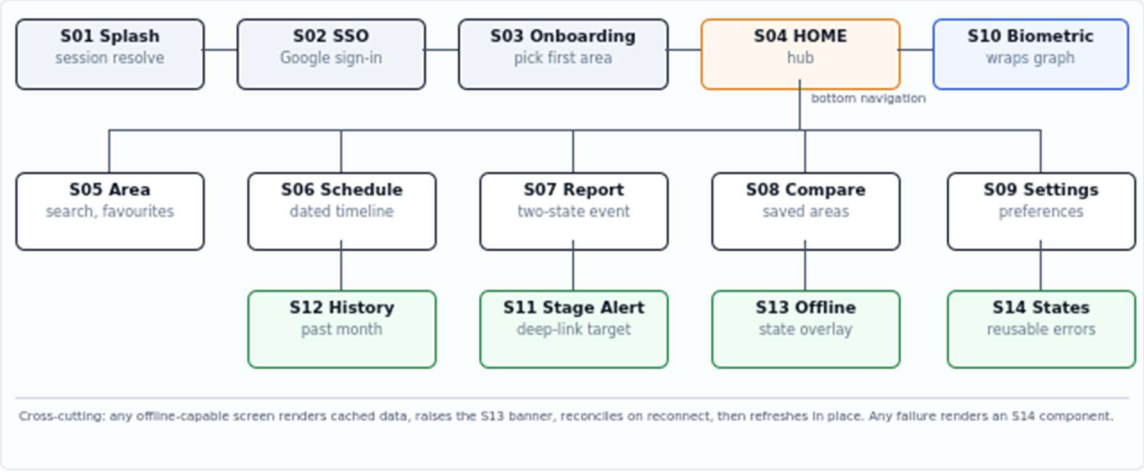


Figure 3. Navigation wireflow. Amber marks the hub, blue the session wrapper, green the Final POE additions that attach to the

5. REST API and Back-End Architecture

5.1 API purpose and responsibilities

The REST API is the only route between the Android client and stored data. It validates the Firebase identity token on every user-scoped call before any business logic runs, and owns CRUD for areas, schedules, reports, favourites and settings. It owns the business logic the client must not hold: deriving current supply state, recalculating schedules when a stage changes, de-duplicating repeat reports and computing the per-area accuracy score. It owns data retrieval, returning narrow JSON payloads sized for expensive mobile data. It owns synchronisation, accepting a batch of queued client reports and answering with an authoritative per-item outcome. Keeping this logic server-side is what keeps four client builds consistent.

5.2 Endpoint specification

Sixteen endpoints are specified in Table 6. They divide into four families. Public read endpoints, area search and area detail, are anonymous, so the catalogue stays cacheable and readable before sign-in. Schedule endpoints return an event window and the derived next event, the latter carrying server time so the client can correct for device clock drift before running the countdown. User-scoped endpoints, covering profile, settings, favourites and device registration, all require a bearer token and are refused with HTTP 401 without one. Report endpoints accept a structured confirmation, expose the per-area accuracy score and accept the offline batch. Every mutating request carries a client-generated identifier, so a retry after a dropped connection returns the original result rather than creating a duplicate; that single decision is what makes the offline queue safe. Verbs follow convention, GET for reads, POST for creation, PUT for replacement and DELETE for removal, and failures return a consistent error object rather than an HTML page. Requests are paged where a collection can grow without bound, so the schedule endpoint takes an explicit window rather than returning everything the server holds, and the history endpoint takes a month. Responses carry a last-updated timestamp so the client can decide whether a cached copy is still worth showing. Rate-sensitive routes, notably report creation, are throttled per user rather than per device, so a reinstall cannot be used to flood the accuracy score.

Verb	Route	Request	Response	Auth
GET	/api/areas?q={text}&page={n}	query text, paging	AreaSummary[]	anonymous
GET	/api/areas/{areaId}	path id	Area	anonymous
GET	/api/schedules?areaId=&from=&to=	area, ISO date range	ScheduleEvent[]	anonymous
GET	/api/schedules/next?areaId=	area id	NextEvent	anonymous
GET	/api/stages/current	none	CurrentStage	anonymous
GET	/api/users/{userId}/settings	path id	UserSettings	bearer
PUT	/api/users/{userId}/settings	UserSettingsUpdate	UserSettings	bearer
GET	/api/users/{userId}/favourites	path id	FavouriteArea[]	bearer
POST	/api/users/{userId}/favourites	{ areaId, label }	201 FavouriteArea	bearer
DELETE	/api/users/{userId}/favourites/{areaId}	path ids	204	bearer

POST	/api/outage-reports	OutageReportCreate	201 OutageReport, or 409	bearer
GET	/api/areas/{areaId}/outage-reports?since=	area, cutoff	OutageReport[]	anonymous
GET	/api/areas/{areaId}/accuracy	path id	AccuracySummary	anonymous
POST	/api/sync/outage-reports	SyncBatch	SyncResult	bearer
POST	/api/notifications/registrations	{ fcmToken, platform }	201	bearer
GET	/api/areas/{areaId}/history?month=	area, month	HistoryDay[]	bearer

Table 6. Endpoint specification. Anonymous read endpoints keep the public catalogue cacheable; every user-scoped route requires a

5.3 Development stack

The client is native Kotlin with Jetpack Compose, Room over SQLite for local persistence, Retrofit and OkHttp for networking, Firebase Authentication for identity and Firebase Cloud Messaging for push. The API is ASP.NET Core Web API hosted on Azure App Service, with Azure SQL as the relational store and Entity Framework Core for data access. The Final POE adds Blob Storage and a NoSQL store, both reached only through the API.

5.4 Online and offline synchronisation

Online, the repository writes straight to the API and mirrors the response into Room. Offline, a confirmation is written to Room as pending and shown to the user as queued. When connectivity returns, a WorkManager job posts the queue to the sync endpoint and the API answers per item with accepted, duplicate or rejected. Server state wins any conflict and the outcome is shown, never silently discarded.

NextEvent (response)	*OutageReportCreate (request)*	*SyncResult (response)*
{ "areaId":"eth-durban-north", "areaName":"Durban North", "currentState":"ON", "stage":2, "nextStartAt":"2026-08-23T20:00:00Z", "nextEndAt":"2026-08-23T22:30:00Z", "serverTime":"2026-08-23T18:17:42Z", "updatedAt":"2026-08-23T18:13:00Z" }	**{ "areaId":"eth-durban-north", "eventType":"POWER_OFF", "reportedAt":"2026-08-23T20:04:11Z", "clientCreatedAt":"2026-08-23T20:04:09Z", "clientReportId":"c9f2-4a11" }** clientReportId makes the POST idempotent, which is what allows a queued report to be retried safely.	**{ "accepted":["c9f2-4a11"], "duplicates":["a71b-9903"], "rejected":[{"id":"b21c-7","reason":"UNKNOWN_AREA"}], "retryAfterSeconds":0 }**

Table 7. Core payload definitions. **serverTime** in NextEvent lets the client correct for device clock drift before running the countdown.

6. UML System Architecture Diagrams

Diagram section. Figure 4 gives the component and deployment view, Figure 5 the sequence view for the offline confirmation flow. Package structure is shown between them. No word allocation applies to this section.

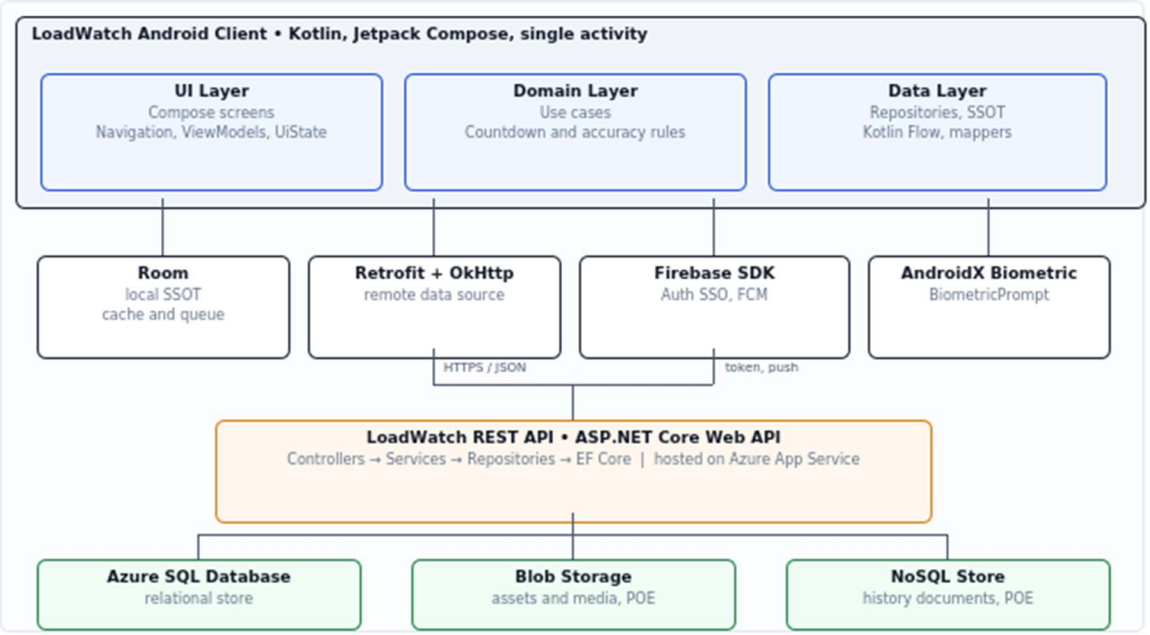


Figure 4. UML component and deployment view. The client reaches cloud storage only through the API, which is what keeps the Final POE blob and NoSQL requirements from leaking into client code.

Package structure implementing Figure 4. Feature packages never import one another; they communicate only through the data layer.

core/ network, database, authentication, notifications, designsystem, common • **data/** repositories, remote data sources, local data sources, mappers • **feature/** auth, home, area, schedule, report, favourites, settings, history, sync

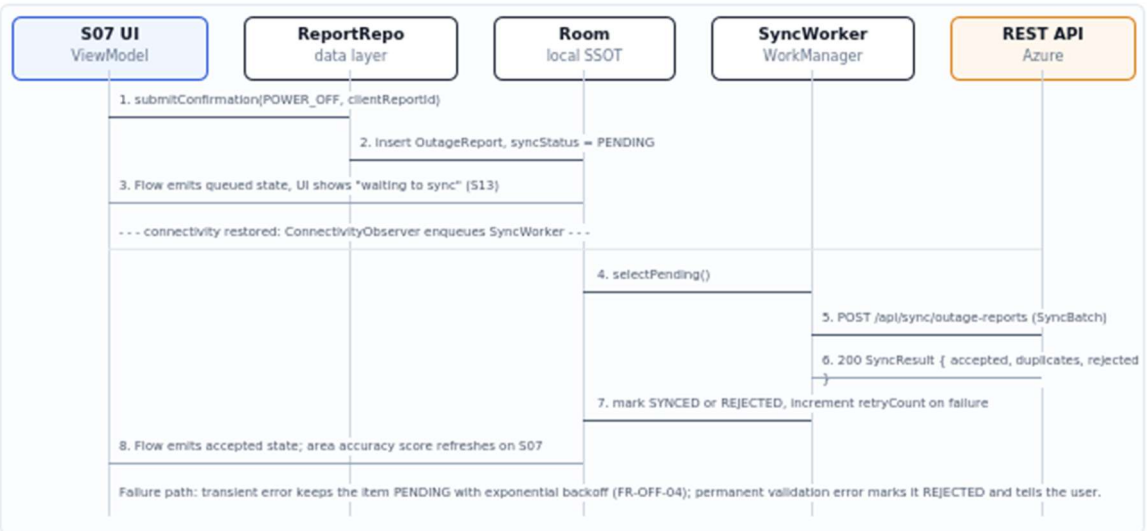


Figure 5. UML sequence diagram: offline community confirmation and reconciliation. Solid lines are calls, grey lines are returns and emissions.

7. Data Models and Schema Definitions

Ten entities have persisted. Types are given as they appear in Kotlin and Room, with the equivalent SQL type noted in Table 8 where the two differ. Identifiers are strings rather than sequential integers throughout, because a client must be able to mint an identifier while offline without colliding with a server sequence; this is the schema decision that makes the offline queue in Section 5.4 workable.

UserProfile holds the identity returned by single sign-on: user identifier, display name, email and creation timestamp. **UserSettings** holds one row per user carrying notification lead time in minutes, enabled alert types, language code and theme preference, which is why settings survive a reinstall. **Area** is the public catalogue entity: area identifier, name, municipality, supplier and coordinates, owned by the server and never written by the client.

ScheduleEvent is one interruption window: event identifier, area foreign key, stage, start timestamp, end timestamp and a last-updated marker used to detect staleness. **OutageReport** is a single community confirmation carrying report identifier, area foreign key, user foreign key, an enumerated event type, the reported timestamp, the client-created timestamp and a sync status. **PendingSync** tracks queued work: local identifier, entity type, payload, attempt count and last attempt time.

FavouriteArea joins a user to an area with a display order. **AccuracySummary** holds an area's computed score, sample size and window. **NotificationRegistration** holds a device token against a user, and **HistoryRecord** is the Final POE document entity.

Relationships are straightforward. UserProfile is the hub for all user-scoped rows; Area is the hub for all schedule and report rows. One user has one settings row, many favourites and many reports; one area has many events, many reports and one accuracy summary. Primary keys are the identifier columns named above, and every foreign key is enforced in Room and in Azure SQL alike.

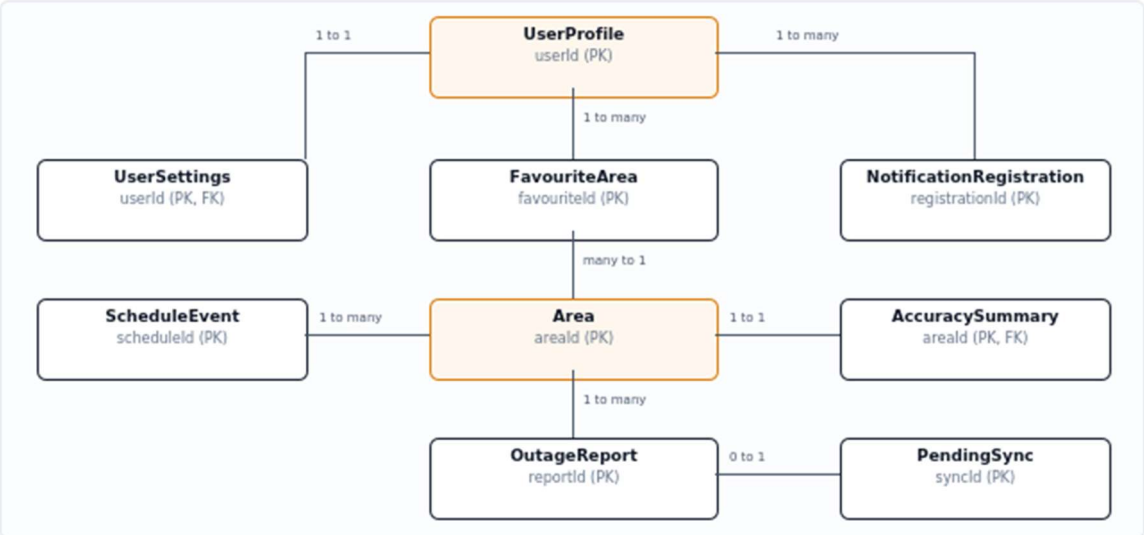


Figure 6. Entity relationship view. Amber marks the two hub entities. OutageReport additionally carries a userId foreign key to UserProfile, omitted from the diagram for legibility.

Entity	Fields and explicit data types	Notes and constraints

Area	areaId: String (PK); name: String; municipality: String; province: String; supplyType: String; latitude: Double?; longitude: Double?; isActive: Boolean	Public catalogue, cached locally in full. supplyType is ESKOM or MUNICIPAL.
ScheduleEvent	scheduleId: String (PK); areaId: String (FK); stage: Int; startAt: Instant; endAt: Instant; source: String; updatedAt: Instant	Instant stored as ISO-8601 UTC text in Room, datetimeoffset in SQL. endAt must exceed startAt.
UserProfile	userId: String (PK); displayName: String; email: String; createdAt: Instant; lastSeenAt: Instant	userId is the Firebase UID, never a local sequence.
UserSettings	userId: String (PK, FK); defaultAreaId: String? (FK); notificationLeadMinutes: Int; notificationsEnabled: Boolean; languageCode: String; biometricEnabled: Boolean; updatedAt: Instant	notificationLeadMinutes constrained to 5 to 120 (FR-SET-02). languageCode is en, zu or af.
FavouriteArea	favouriteId: String (PK); userId: String (FK); areaId: String (FK); label: String; sortOrder: Int; createdAt: Instant	Unique on (userId, areaId). Deleting a favourite never cascades to Area (FR-FAV-03).
OutageReport	reportId: String (PK); clientReportId: String; userId: String (FK); areaId: String (FK); eventType: Enum {POWER_OFF, POWER_BACK_ON}; reportedAt: Instant; clientCreatedAt: Instant; syncStatus: Enum {PENDING, SYNCED, RETRYING, REJECTED}	clientReportId is unique per user and makes the POST idempotent, enabling safe retry.
AccuracySummary	areaId: String (PK, FK); accuracyScore: Decimal(3,2); sampleSize: Int; windowHours: Int; calculatedAt: Instant	Score between 0.00 and 1.00, recomputed on report ingestion.
PendingSync	syncId: String (PK); entityType: String; entityId: String; payloadJson: String; retryCount: Int; nextAttemptAt: Instant; status: Enum {PENDING, RETRYING, REJECTED}; lastError: String?	Device only, never synchronised to the server. Generic by design so future entities reuse the queue.
NotificationRegistration	registrationId: String (PK); userId: String (FK); fcmToken: String; platform: String; lastSeenAt: Instant	Rotated when FCM issues a new token; stale rows pruned server-side.
HistoryDay	areaId: String; date: LocalDate; scheduledWindows: List<TimeRange>; observedWindows: List<TimeRange>; matched: Boolean	Final POE. Read-heavy and append-only, so served from the NoSQL store (FR-NOSQL-01).

Table 8. Data models with explicit data types for every field.

8. Project Management Plan

Plan section. Figure 7 is the sixteen-week Gantt chart with named owners, three milestones and two dedicated testing gates; Table 9 is the risk register. No word allocation applies to this section.

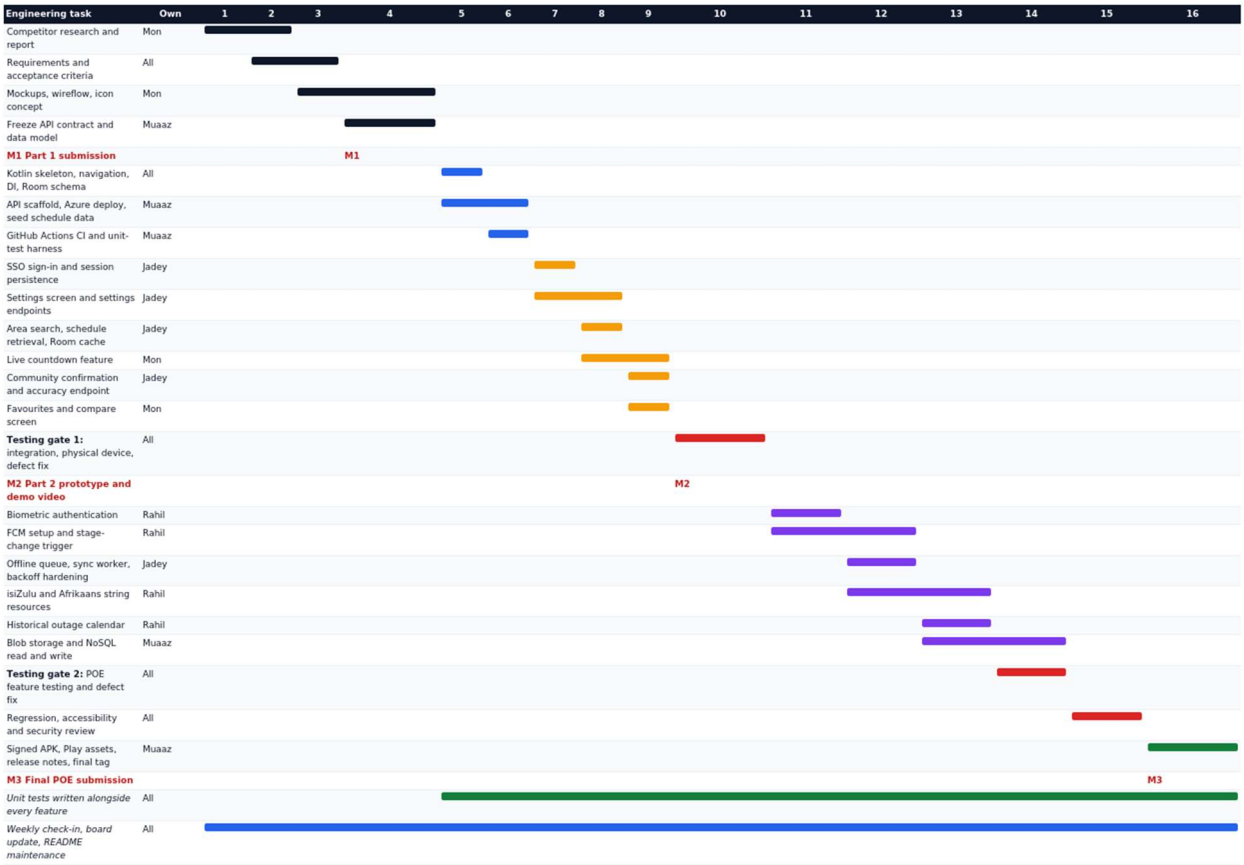


Figure 7. Sixteen-week Gantt chart. Dark blue is research and design, mid blue is infrastructure, amber is Part 2 feature work, purple is Final POE feature work, red is dedicated testing and bug fixing, green is continuous quality and release.

Risk	Likelihood	Impact	Mitigation built into the plan
API contract changes after client work begins	Medium	High	Contract frozen in week 4 before any client code (Table 6 and 7); changes require a versioned route.
No physical Android device available for marking	Low	High	Device secured by week 5; every testing gate is run on hardware, not an emulator .
Schedule data source unavailable or drifts	Medium	Medium	Seeded schedule data per area and stage from week 5, so no feature depends on an external publisher.
Final POE features force rework of Part 2 code	Medium	High	Extension-point interfaces defined in Part 2 (Section 8); Final POE work adds implementations only.
Uneven commit history across members	Medium	Medium	One vertical slice per member (Table 4), branch per feature, weekly board review against the Gantt.

Table 9. Risk register with mitigations that are already scheduled rather than aspirational.

9. Conclusion

This document turns the research findings into a modular, implementable blueprint. The architecture separates interface, domain and data behind a repository-mediated single source of truth, so no screen needs to know whether data arrived from the network or from Room. The API is specified down to

payloads and endpoint families, every screen the product will need is already drawn, and every requirement carries an acceptance criterion and a named owner.

Part 2 will deliver single sign-on, the settings screen, the hosted REST API and the three user-defined features against the contracts fixed here. The Final POE extends the same foundations with biometrics, offline synchronisation, push, localisation, the historical calendar and blob and NoSQL storage. Nothing in that list requires Part 2 code to be rewritten, because each of those capabilities was given an interface and a screen in this document before any of them was scheduled. That was the design goal, and it is what the traceability matrix in Section 3 exists to prove.

References

[1] R. Fielding, M. Nottingham, and J. Reschke, Eds., "HTTP Semantics," RFC 9110, Internet Engineering Task Force, Jun. 2022. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9110. Accessed: Aug. 24, 2026.

[2] W3C, "Web Content Accessibility Guidelines (WCAG) 2.2," W3C Recommendation, Oct. 5, 2023. [Online]. Available: https://www.w3.org/TR/WCAG22/. Accessed: Aug. 24, 2026.

[3] Google Play, "ESP 'EskomSePush'," wellwellwell. [Online]. Available: https://play.google.com/store/apps/details?id=com.ashwhale.sepush.eskom. Accessed: Aug. 23, 2026.

[4] EskomSePush, "ESP: South Africa's #1 Loadshedding and Community App." [Online]. Available: https://esp.info/. Accessed: Aug. 23, 2026.

[5] EskomSePush, "EskomSePush API Subscription: Documentation v3.0 and v3.1." [Online]. Available: https://eskomsepush.gumroad.com/l/api. Accessed: Aug. 23, 2026.

[7] Google Play, "Load Shedding Notifier," abisoft. [Online]. Available: https://play.google.com/store/apps/details?id=com.abisoft.loadsheddingnotifier. Accessed: Aug. 23, 2026.

[9] Acumen Software, "My Smart City: Mobile Smart City Solution." [Online]. Available: https://www.mysmart.city/. Accessed: Aug. 23, 2026.

[11] Android Developers, "Guide to app architecture." [Online]. Available: https://developer.android.com/topic/architecture. Accessed: Aug. 23, 2026.

[12] Android Developers, "Save data in a local database using Room." [Online]. Available: https://developer.android.com/training/data-storage/room. Accessed: Aug. 23, 2026.

[13] Firebase, "Authenticate with Google on Android." [Online]. Available: https://firebase.google.com/docs/auth/android/google-signin. Accessed: Aug. 23, 2026.

[14] Firebase, "Set up a Firebase Cloud Messaging client app on Android." [Online]. Available: https://firebase.google.com/docs/cloud-messaging/android/client. Accessed: Aug. 23, 2026.

[15] Android Developers, "Show a biometric authentication dialog." [Online]. Available: https://developer.android.com/identity/sign-in/biometric-auth. Accessed: Aug. 23, 2026.

[16] Android Developers, "Schedule tasks with WorkManager." [Online]. Available: https://developer.android.com/topic/libraries/architecture/workmanager. Accessed: Aug. 23, 2026.

[17] Microsoft Learn, "Tutorial: Create a controller-based web API with ASP.NET Core." [Online]. Available: https://learn.microsoft.com/en-us/aspnet/core/tutorials/first-web-api. Accessed: Aug. 23, 2026.

[18] Microsoft Learn, "Quickstart: Deploy an ASP.NET web app to Azure App Service." [Online]. Available: https://learn.microsoft.com/en-us/azure/app-service/quickstart-dotnetcore. Accessed: Aug. 23, 2026.